

Notebook Aula 02: Representação de Dados e Estruturas em Python (Profª Kadidja)

Tópico 1: Fundamentos — Dados na Memória e Referências (`id()`)

Em Python, as variáveis funcionam como **referências** para objetos alocados na memória RAM. A função nativa `id()` permite observar o identificador único associado a esse objeto durante sua existência.

```
1 # =====
2 # TÓPICO 1: Dados na Memória
3 # Demonstração de referências de variáveis e busca do ID único na memória RAM
4 # =====
5
6 # Criação de um objeto do tipo inteiro (20) e atribuição à variável 'idade'
7 idade = 20
8
9 # Exibe o valor contido e o endereço único de memória atribuído pelo Python
10 # print(f"Valor inicial da variável 'idade': {idade}")
11 print("Valor inicial da variável 'idade'", idade)
12 print(f"Endereço/ID na memória RAM (id(idade)): {id(idade)}")
13
14 # Ao reatribuir um novo valor, Python cria um novo objeto na memória
15 idade = 21
16 print(f"\nNovo valor atribuído a 'idade': {idade}")
17 print(f"Novo endereço/ID na memória RAM: {id(idade)}")
```

Valor inicial da variável 'idade' 20
Endereço/ID na memória RAM (id(idade)): 11645960

Novo valor atribuído a 'idade': 21
Novo endereço/ID na memória RAM: 11645992

Tópico 2: Estruturas Sequenciais — Listas e Indexação

A **Lista** (`list`) é a principal estrutura sequencial em Python, substituindo os vetores/arrays tradicionais. O acesso ocorre por **índices positivos** (a partir de `0`) e por **índices negativos** para navegação reversa (sendo `-1` o último elemento).

```
1 # =====
2 # TÓPICO 2: Estruturas Sequenciais (Listas e Indexação)
3 # Acesso a elementos via índices positivos (0 até N-1) e reverso (-1 até -N)
4 # =====
5
6 # Definição de uma lista com elementos inteiros sequenciais
7 numeros = [10, 20, 30, 40, 50]
8
9 print("--- Acesso por Índices Positivos ---")
10 print(f"Primeiro elemento [índice 0]: {numeros[0]}")
11 print(f"Segundo elemento [índice 1]: {numeros[1]}")
12 print(f"Terceiro elemento [índice 2]: {numeros[2]}")
13
14 print("\n--- Acesso por Índices Negativos (Reverso) ---")
15 print(f"Último elemento [índice -1]: {numeros[-1]}")
16 print(f"Penúltimo elemento [índice -2]: {numeros[-2]}")
17 print(f"Primeiro via reverso [índice -5]: {numeros[-5]}")
```

--- Acesso por Índices Positivos ---
Primeiro elemento [índice 0]: 10
Segundo elemento [índice 1]: 20
Terceiro elemento [índice 2]: 30

--- Acesso por Índices Negativos (Reverso) ---
Último elemento [índice -1]: 50
Penúltimo elemento [índice -2]: 40
Primeiro via reverso [índice -5]: 10

Tópico 3: Manipulação Dinâmica e Percurso em Listas

As listas em Python são dinâmicas e aceitam inserções com o método `.append()`. Para percorrer os elementos, podemos utilizar o percurso **por valor** ou **por posição (índice)**.

```
1 # =====
2 # TÓPICO 3: Manipulação Dinâmica e Percurso em Listas
3 # Inserção com .append() e iteração por valor vs por posição/índice
4 # =====
5
6 # Criação de uma lista inicial com três notas
7 notas = [8.5, 7.0, 9.2]
8
9 # Adicionando um novo elemento dinamicamente ao final da lista
10 notas.append(8.0)
11 print(f"Lista de notas após adição dinâmica: {notas}\n")
12
13 # 1. PERCURSO POR VALOR
14 # Recomendado quando não precisamos manipular ou saber o índice numérico
15 print("--- Percurso por Valor (for nota in notas) ---")
16 for nota in notas:
17     print(f"Nota lida: {nota}")
18
19 # 2. PERCURSO POR POSIÇÃO/ÍNDICE
20 # Recomendado quando a posição exata (índice) é necessária para processamento
21 print("\n--- Percurso por Posição (for i in range(len(notas))) ---")
22 for i in range(len(notas)):
23     print(f"Posição/Índice {i} -> Nota: {notas[i]}")
```

Lista de notas após adição dinâmica: [8.5, 7.0, 9.2, 8.0]

--- Percurso por Valor (for nota in notas) ---

Nota lida: 8.5

Nota lida: 7.0

Nota lida: 9.2

Nota lida: 8.0

--- Percurso por Posição (for i in range(len(notas))) ---

Posição/Índice 0 -> Nota: 8.5

Posição/Índice 1 -> Nota: 7.0

Posição/Índice 2 -> Nota: 9.2

Posição/Índice 3 -> Nota: 8.0

✓ Tópico 4: A 2ª Dimensão — Matrizes e Loops Aninhados

Uma matriz bidimensional é representada em Python utilizando uma **lista de listas**. A navegação em duas dimensões é realizada através de dois loops `for` aninhados: o externo percorre as **linhas** e o interno as **colunas**.

```
1 # =====
2 # TÓPICO 4: Matrizes (Estruturas Multidimensionais)
3 # Representação através de listas de listas e varredura com loops aninhados
4 # =====
5
6 # Definindo uma matriz 3x3
7 matriz = [
8     [1, 2, 3], # Linha 0
9     [4, 5, 6], # Linha 1
10    [7, 8, 9]  # Linha 2
11 ]
12
13 # Acesso direto a um elemento específico: matriz[linha][coluna]
14 # Acessando linha 1, coluna 2 -> elemento de valor 6
15 print(f"Elemento na posição matriz[1][2]: {matriz[1][2]}\n")
16
17 # Varredura completa da matriz com loops aninhados
18 print("--- Varredura Completa da Matriz (Linhas x Colunas) ---")
19 for i in range(len(matriz)): # 1º loop: itera pelas linhas
20     for j in range(len(matriz[i])): # 2º loop: itera pelas colunas da linha atual
21         print(f"matriz[{i}][{j}] = {matriz[i][j]}", end=" | ")
22     print() # Quebra de linha ao final de cada linha da matriz
```

Elemento na posição matriz[1][2]: 6

--- Varredura Completa da Matriz (Linhas x Colunas) ---

matriz[0][0] = 1 | matriz[0][1] = 2 | matriz[0][2] = 3 |

matriz[1][0] = 4 | matriz[1][1] = 5 | matriz[1][2] = 6 |

matriz[2][0] = 7 | matriz[2][1] = 8 | matriz[2][2] = 9 |

Tópico 5: Modelagem de Entidades com Dicionários (dict)

Dicionários associam **chaves** (str) a **valores** de qualquer tipo (números, strings, listas). Essa estrutura permite modelar objetos e entidades do mundo real com facilidade de acesso através das chaves.

```
1 # =====
2 # TÓPICO 5: Representando Entidades com Dicionários (dict)
3 # Mapeamento chave -> valor para representação estruturada de dados
4 # =====
5
6 # Modelando a entidade 'Aluno' usando um dicionário
7 aluno = {
8     "nome": "João",
9     "idade": 20,
10    "notas": [8.5, 9.0]
11 }
12
13 # Acessando e exibindo os atributos através de suas chaves descritivas
14 print(f"Nome do Aluno: {aluno['nome']}")
15 print(f"Idade do Aluno: {aluno['idade']}")
16 print(f"Notas do Aluno: {aluno['notas']}")
17
18 # Operação com estrutura interna (lista de notas dentro do dicionário)
19 media_aluno = sum(aluno['notas']) / len(aluno['notas'])
20 print(f"Média calculada: {media_aluno:.2f}")
```

```
Nome do Aluno: João
Idade do Aluno: 20
Notas do Aluno: [8.5, 9.0]
Média calculada: 8.75
```

✓ Tópico 6: Modelagem Avançada com dataclass

A abordagem com @dataclass aproxima o Python de estruturas fortemente tipadas (structs). Ela exige a definição explícita de atributos e seus respectivos tipos, trazendo maior robustez e clareza ao código.

```
1 # =====
2 # TÓPICO 6: Modelagem Avançada com dataclass
3 # Uso de orientação a objetos com tipagem explícita de atributos
4 # =====
5
6 from dataclasses import dataclass
7 from typing import List
8
9 # Definição do molde da entidade Aluno usando a anotação @dataclass
10 @dataclass
11 class Aluno:
12     nome: str
13     idade: int
14     notas: List[float]
15
16 # Instanciando o objeto a partir da classe definida
17 aluno1 = Aluno(nome="Ana", idade=21, notas=[9.0, 9.5])
18
19 # Acesso direto aos atributos utilizando notação por ponto (.atributo)
20 print(f"Instância de Aluno: {aluno1}")
21 print(f"Nome do Aluno: {aluno1.nome}")
22 print(f"Primeira nota: {aluno1.notas[0]}")
23
24 # Cálculo da média
25 media_ana = sum(aluno1.notas) / len(aluno1.notas)
26 print(f"Média de {aluno1.nome}: {media_ana:.2f}")
```

```
Instância de Aluno: Aluno(nome='Ana', idade=21, notas=[9.0, 9.5])
Nome do Aluno: Ana
Primeira nota: 9.0
Média de Ana: 9.25
```

✓ Tópico 7: Estudo de Caso — Sistema de Cadastro de Alunos

Aplicações Práticas:

1. Inicializar lista mestre vazia `alunos = []`.
2. Cadastrar 3 alunos (nome, idade, matrícula, notas).

3. Exibir os registros cadastrados na tela.
4. Calcular a média aritmética das notas de cada aluno.
5. Identificar via código o aluno com a maior média.

```
1 # =====
2 # TÓPICO 7: Estudo de Caso – Sistema de Cadastro de Alunos
3 # Solução Arquitetural: Lista de Dicionários / Objetos
4 # =====
5
6 # Passo 1: Lista mestre vazia para armazenar todos os alunos cadastrados
7 alunos = []
8
9 # Passo 2: Definindo as entidades individualmente
10 aluno1 = {
11     "nome": "Carlos Silva",
12     "idade": 20,
13     "matricula": "202401",
14     "notas": [7.5, 8.0, 9.0]
15 }
16
17 aluno2 = {
18     "nome": "Mariana Souza",
19     "idade": 22,
20     "matricula": "202402",
21     "notas": [9.5, 9.0, 10.0]
22 }
23
24 aluno3 = {
25     "nome": "Beatriz Lima",
26     "idade": 19,
27     "matricula": "202403",
28     "notas": [6.0, 7.5, 8.0]
29 }
30
31 # Adicionando os objetos modelados à lista mestre
32 alunos.append(aluno1)
33 alunos.append(aluno2)
34 alunos.append(aluno3)
35
36 print("=== PASSO 3 & 4: LISTAGEM DE REGISTROS E CÁLCULO DE MÉDIAS ===\n")
37
38 # Variáveis auxiliares para controle do aluno de maior média
39 maior_media = -1.0
40 aluno_com_maior_media = None
41
42 # Percorrendo a lista principal de alunos
43 for aluno in alunos:
44     nome = aluno["nome"]
45     matricula = aluno["matricula"]
46
47     # Isola a lista interna de notas para processamento
48     notas_atuais = aluno["notas"]
49
50     # Cálculo da média aritmética do aluno atual
51     media = sum(notas_atuais) / len(notas_atuais)
52
53     print(f"Matrícula: {matricula} | Nome: {nome:<15} | Notas: {notas_atuais} | Média: {media:.2f}")
54
55     # Passo 5: Comparação para encontrar a maior média
56     if media > maior_media:
57         maior_media = media
58         aluno_com_maior_media = aluno
59
60 print("\n" + "="*60)
61 print("=== PASSO 5: ALUNO IDENTIFICADO COM A MAIOR MÉDIA ===")
62 print("="*60)
63 if aluno_com_maior_media:
64     print(f"Nome do Aluno: {aluno_com_maior_media['nome']}")
65     print(f"Matrícula: {aluno_com_maior_media['matricula']}")
66     print(f"Maior Média: {maior_media:.2f}")
```

=== PASSO 3 & 4: LISTAGEM DE REGISTROS E CÁLCULO DE MÉDIAS ===

Matrícula: 202401	Nome: Carlos Silva	Notas: [7.5, 8.0, 9.0]	Média: 8.17
Matrícula: 202402	Nome: Mariana Souza	Notas: [9.5, 9.0, 10.0]	Média: 9.50
Matrícula: 202403	Nome: Beatriz Lima	Notas: [6.0, 7.5, 8.0]	Média: 7.17

=====

=== PASSO 5: ALUNO IDENTIFICADO COM A MAIOR MÉDIA ===

=====

✓ Tópico 8: Desafio Prático — Gestão de Estoque

Requisitos do Desafio:

1. Criar entidade **Produto** contendo `nome`, `preco` e `quantidade`.
2. Criar uma lista principal cadastrando **pelo menos 5 produtos**.
3. **Métricas Solicitadas:**
 - Calcular o valor total financeiro em estoque (`preco * quantidade`).
 - Identificar o produto **mais caro** (maior preço unitário).
 - Identificar o produto com **maior volume armazenado** (maior quantidade).

```
1 # =====
2 # TÓPICO 8: Desafio Prático — Gestão de Estoque
3 # Lista de produtos com processamento de métricas e busca de extremos
4 # =====
5
6 # Requisito 1 & 2: Cadastro de pelo menos 5 produtos usando lista de dicionários
7 estoque = [
8     {"nome": "Notebook Pro", "preco": 4500.00, "quantidade": 8},
9     {"nome": "Mouse Sem Fio", "preco": 120.00, "quantidade": 45},
10    {"nome": "Teclado Mecânico", "preco": 350.00, "quantidade": 25},
11    {"nome": "Monitor 27 Ultra", "preco": 1800.00, "quantidade": 12},
12    {"nome": "Cadeira Ergonômica", "preco": 1200.00, "quantidade": 15}
13 ]
14
15 # Inicialização de variáveis acumuladoras e de referência para os extremos
16 valor_total_estoque = 0.0
17 produto_mais_caro = estoque[0]
18 produto_maior_volume = estoque[0]
19
20 print("=== RELATÓRIO DETALHADO DO ESTOQUE CADASTRADO ===\n")
21
22 # Processamento dos itens do estoque
23 for produto in estoque:
24     # Cálculo do valor financeiro total do item (Preço * Quantidade)
25     subtotal_item = produto["preco"] * produto["quantidade"]
26
27     # Acumula o valor total do estoque
28     valor_total_estoque += subtotal_item
29
30     # Impressão formatada dos dados de cada produto
31     print(f"Produto: {produto['nome']:<20} | "
32           f"Preço Un.: R$ {produto['preco']:8.2f} | "
33           f"Qtd: {produto['quantidade']:3d} | "
34           f"Subtotal: R$ {subtotal_item:9.2f}")
35
36     # Verificação 1: Produto de maior preço unitário
37     if produto["preco"] > produto_mais_caro["preco"]:
38         produto_mais_caro = produto
39
40     # Verificação 2: Produto com maior quantidade armazenada
41     if produto["quantidade"] > produto_maior_volume["quantidade"]:
42         produto_maior_volume = produto
43
44 print("\n" + "="*65)
45 print("=== MÉTRICAS E RESULTADOS DO DESAFIO DE ESTOQUE ===")
46 print("="*65)
47 print(f"1. Valor Total Financeiro em Estoque: R$ {valor_total_estoque:,.2f}")
48 print(f"2. Produto Mais Caro (Preço Un.): {produto_mais_caro['nome']} (R$ {produto_mais_caro['preco']:8.2f})")
49 print(f"3. Maior Volume de Estoque (Qtd): {produto_maior_volume['nome']} ({produto_maior_volume['quantidade']} unid:

=== RELATÓRIO DETALHADO DO ESTOQUE CADASTRADO ===

Produto: Notebook Pro          | Preço Un.: R$ 4500.00 | Qtd: 8 | Subtotal: R$ 36000.00
Produto: Mouse Sem Fio        | Preço Un.: R$ 120.00  | Qtd: 45 | Subtotal: R$ 5400.00
Produto: Teclado Mecânico     | Preço Un.: R$ 350.00  | Qtd: 25 | Subtotal: R$ 8750.00
Produto: Monitor 27 Ultra     | Preço Un.: R$ 1800.00 | Qtd: 12 | Subtotal: R$ 21600.00
Produto: Cadeira Ergonômica   | Preço Un.: R$ 1200.00 | Qtd: 15 | Subtotal: R$ 18000.00

=====
=== MÉTRICAS E RESULTADOS DO DESAFIO DE ESTOQUE ===
=====
```

- 1. Valor Total Financeiro em Estoque: R\$ 89,750.00
- 2. Produto Mais Caro (Preço Un.): Notebook Pro (R\$ 4500.00)
- 3. Maior Volume de Estoque (Qtd): Mouse Sem Fio (45 unidades)